

Modulo 2

Error 404

2026-04-08

#1 Generar un vector secuencia

Generar una secuencia de números que de dos en dos entre 1 y 100.000.

```
# 1. Usando un comando específico de R
met_seq <- function() {
  secuencia <- seq(from = 1, to = 100000, by = 2)
  return(secuencia)
}
```

```
# 2. Usando un bucle 'for' (Manual)

met_for <- function() {
  n <- 50000 # Sabemos que habrá 50,000 elementos
  secuencia <- numeric(n)
  valor <- 1
  for (i in 1:n) {
    secuencia[i] <- valor
    valor <- valor + 2
  }
  return(secuencia)
}
```

```
# COMPARACIÓN
cat("Tiempo para el comando específico (seq):\n")
```

```
## Tiempo para el comando específico (seq):
```

```
system.time(met_seq())
```

```
##    user  system elapsed
## 0.002   0.000   0.001
```

```
cat("\nTiempo para el bucle 'for':\n")
```

```
##
```

```
## Tiempo para el bucle 'for':
```

```
system.time(met_for())
```

```
##    user  system elapsed
## 0.008   0.000   0.008
```

#2 Implementación de una serie Fibonachi o Fibonacci

¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```

fib_limite <- function(limite) {
  fib <- c(0, 1)
  iteracion <- 2

  while (fib[length(fib)] <= limite) {
    nuevo_numero <- fib[length(fib)] + fib[length(fib) - 1]
    fib <- c(fib, nuevo_numero)
    iteracion <- iteracion + 1
  }

  return(list(
    iteraciones = iteracion,
    ultimo_numero = fib[length(fib)],
    serie_completa = fib
  ))
}

resultado <- fib_limite(1000000)

cat("Resultados de la búsqueda:\n")

## Resultados de la búsqueda:
cat("-----\n")

## -----
cat("Número de iteraciones (n):", resultado$iteraciones, "\n")

## Número de iteraciones (n): 32
cat("Primer valor que supera el millón:", resultado$ultimo_numero, "\n")

## Primer valor que supera el millón: 1346269

#3 Definición matemática recurrente
¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

recurrente <- c(0, 1)

tiempo <- system.time({

  while (recurrente[length(recurrente)] <= 1000000) {
    n <- length(recurrente)
    nuevo_termino <- recurrente[n] + recurrente[n-1]
    recurrente <- c(recurrente, nuevo_termino)
  }
})

# RESULTADOS
cat("--- Resultados de la Serie ---\n")

## --- Resultados de la Serie ---
cat("Último número generado:", recurrente[length(recurrente)], "\n")

## Último número generado: 1346269

```

```
cat("Cantidad de iteraciones (n):", length(recurrente), "\n")
```

```
## Cantidad de iteraciones (n): 32
```

```
cat("\n--- Performance ---\n")
```

```
##
```

```
## --- Performance ---
```

```
print(tiempo)
```

```
##      user  system elapsed
```

```
## 0.003   0.000   0.003
```

#4 Ordenación de un vector por método burbuja

Compara la performance de ordenación del método burbuja vs el método sort de R

```
library(microbenchmark)
```

```
x <- sample(1:20000, 20000)
```

```
bubble <- function(x) {  
  it <- length(x)  
  for (i in 1:(it - 1)) {  
    for (j in 1:(it - i)) {  
      if (x[j] > x[j + 1]) {  
        temp <- x[j]  
        x[j] <- x[j + 1]  
        x[j + 1] <- temp  
      }  
    }  
  }  
  return(x)  
}
```

```
resultado <- microbenchmark(  
  bubble = bubble(x),  
  sort_R = sort(x),  
  times = 3  
)
```

```
summary(resultado)
```

```
##      expr      min       lq      mean   median      uq  
## 1 bubble 20659150.014 2.069260e+07 2.083266e+07 20726047.1 20919407.91  
## 2 sort_R    213.971 2.991355e+02 3.534503e+02    384.3    423.19  
##      max neval  
## 1 21112768.74     3  
## 2     462.08     3
```

#5 La penitencia de Newton

Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta $1 * 1010$ y verificar cuál de los dos es más eficiente.

```
#Método 1: Suma con for
```

```
met_for <- function() {
```

```

num <- 1:1010
sum <- 0
for (i in num) {
  sum <- sum + i
}
return(sum)
}

#Metodo 2: Suma con formula
met_formula <- function() {
  numeros <- 1:1010
  primero <- numeros[1]
  ultimo <- numeros[length(numeros)]
  n <- length(numeros)
  suma <- (primero + ultimo) * n / 2
  return(suma)
}

#Comparacion
tiempo_for <- system.time({
  result_for <- met_for()
})

tiempo_formula <- system.time({
  result_formula <- met_formula()
})

print(result_for)

## [1] 510555

print(result_formula)

## [1] 510555

print(tiempo_for)

##      user  system elapsed
## 0.002   0.000   0.003

print(tiempo_formula)

##      user  system elapsed
##      0      0      0

```